Homework2

February 4, 2019

```
In [1]: import numpy as np
        import matplotlib.pyplot as plt
        from scipy import stats
        from scipy.stats import norm
        import sklearn.linear_model
        from sklearn.linear_model import Lasso

        import pandas as pd
        import sklearn

        from sklearn.linear_model import lasso_path, enet_path
        from itertools import cycle

        import matplotlib.pyplot as plt
```

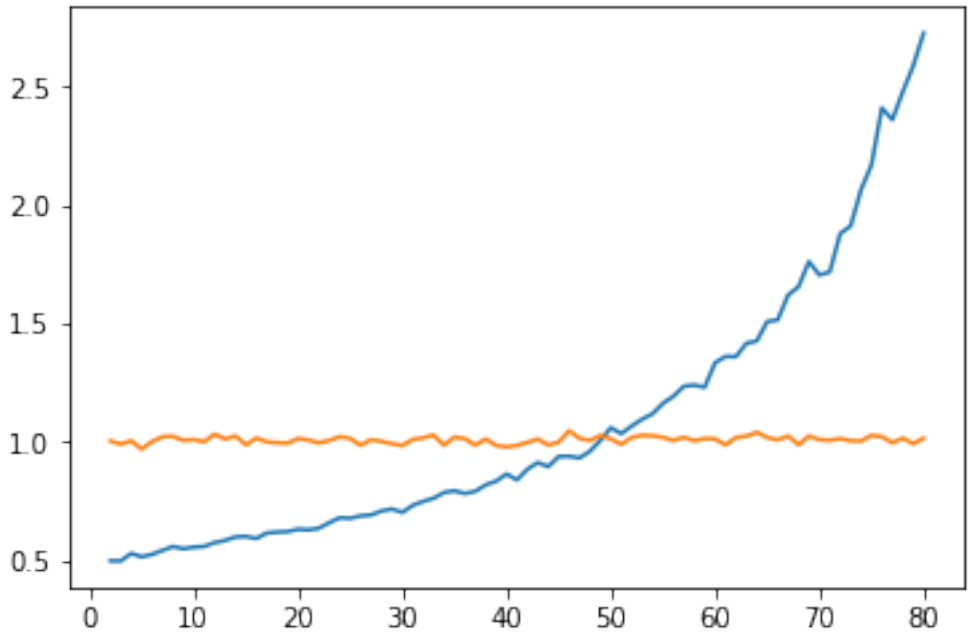
```
In [2]: import warnings
        warnings.filterwarnings('ignore')
```

1 Problem 1: Prediction error and dimension.

Use regular linear regression, average squared prediction errors increase with p, while with Lasso regression, average squared prediction errors remain almost the same, which is approximately one.

```
In [5]: def RV_generator(p):
            n = 100
            X = np.random.randn(n, p)
            Y = X[:, 0] * 1/np.sqrt(2) + np.random.randn(n)/np.sqrt(2)
            return X, Y

        avg_mse = []
        avg_mse_lasso = []
        for p in range(2,81):
            mse = 0
            mse_lasso = 0
            for i in range(100):
                X,Y = RV_generator(p)
```

```
            X_test,Y_test = RV_generator(p)
            model = sklearn.linear_model.LinearRegression()
            model.fit(X,Y)
            Y_pred = model.predict(X_test)
            mse += sklearn.metrics.mean_squared_error(Y_test, Y_pred) / 100

            model_lasso = Lasso(alpha = np.sqrt(2*np.log(p)/100), normalize=True, fit_inte
            model_lasso.fit(X, Y)
            # Then call model.predict, etc.
            Y_pred_lasso = model_lasso.predict(X_test)
            mse_lasso += sklearn.metrics.mean_squared_error(Y_test, Y_pred_lasso) / 100

        avg_mse.append(mse)
        avg_mse_lasso.append(mse_lasso)

plt.plot(range(2,81),avg_mse)
plt.plot(range(2,81),avg_mse_lasso)
plt.show()
```



In [ ]:

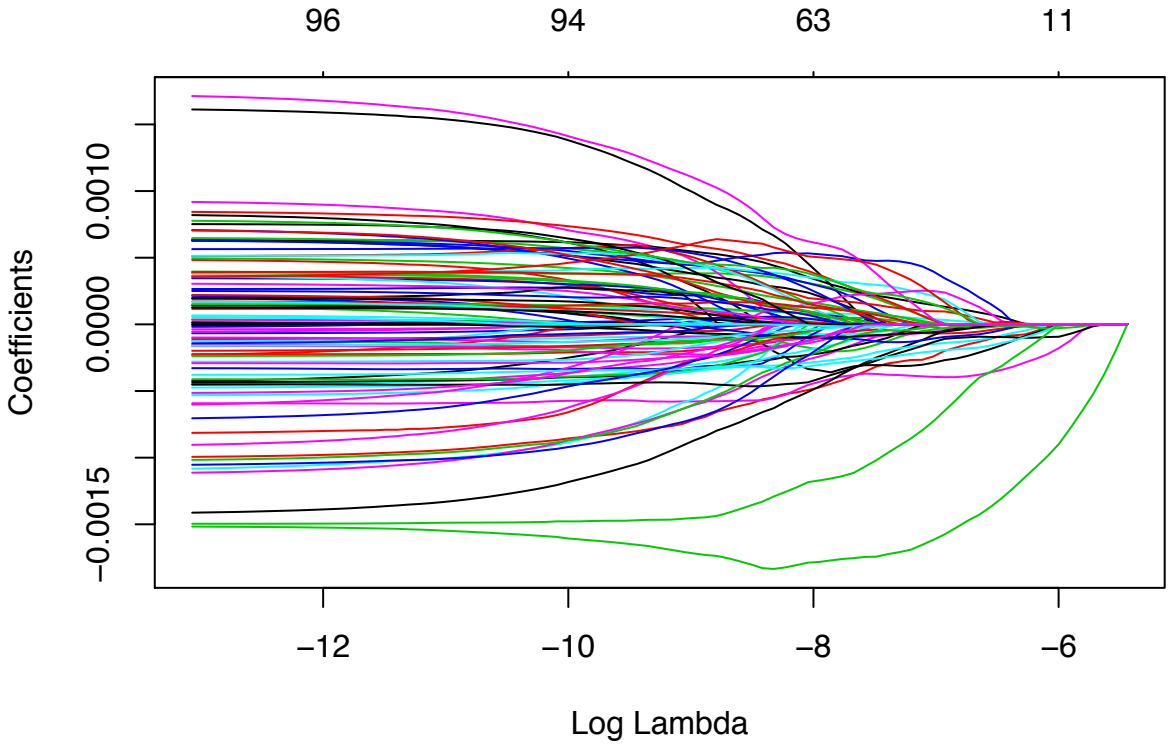
Problem 2

Problem 2 (a)

```
data = read.table('trends_train.csv', sep = ",",header=TRUE)
data$Week = as.Date(as.character(data$Week), format = "%Y-%m-%d")
#class(data$Week)
search.vol = data.frame(data[,2:(ncol(data)-1)])
#check
colnames(search.vol[96]) == colnames(data[97])
```

```
## [1] TRUE
```

```
fit_lasso = glmnet(as.matrix(search.vol),as.matrix(data$logret),family='gaussian',alpha=1)
#fit_lasso$lambda
plot(fit_lasso,xvar='lambda')
```

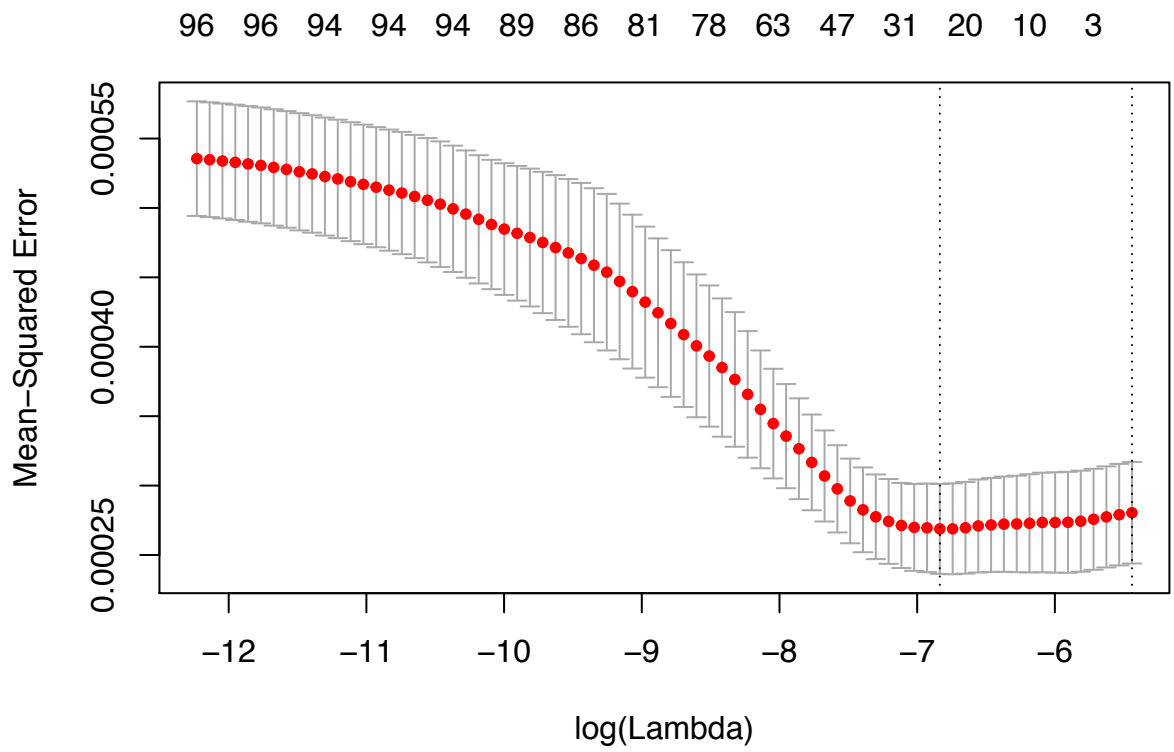


(b)

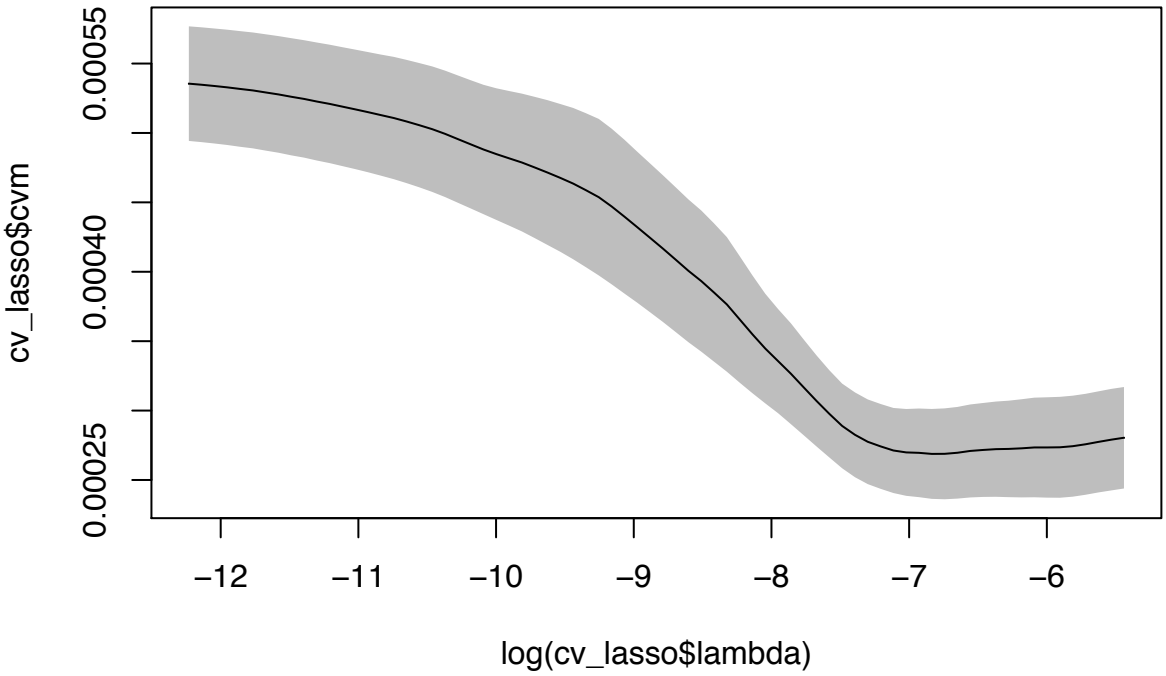
```
cv_lasso = cv.glmnet(as.matrix(search.vol),as.matrix(data$logret),family='gaussian',alpha=1)
names(cv_lasso)
```

```
##  [1] "lambda"     "cvm"        "cvsd"       "cvup"       "cvlo"
##  [6] "nzero"      "name"       "glmnet.fit" "lambda.min" "lambda.1se"
```

```
plot(cv_lasso)
```




```
#We can pull out the mean, sd, upper, and lower bounds with
cmv = cv_lasso$cvm
cvup = cv_lasso$cvup
cvlo = cv_lasso$cvlo
#so we can make pretty custom plots, like
plot(log(cv_lasso$lambda),cv_lasso$cvm,ylim=c(min(cv_lasso$cvlo),max(cv_lasso$cvup)),type='l')
polygon(c(log(cv_lasso$lambda),rev(log(cv_lasso$lambda))),c(cv_lasso$cvup,rev(cv_lasso$cvlo)),col='grey
lines(log(cv_lasso$lambda),cv_lasso$cvm)
```



```
cv_lasso = cv.glmnet(as.matrix(search.vol),as.matrix(data$logret),family='gaussian',alpha=1)

#we can also directly access the chosen values of lambda
Lambda.min = cv_lasso$lambda.min
Lambda.1se = cv_lasso$lambda.1se

#Number of predictor when Lambda is the minimum
sum(as.logical(coef(fit_lasso,s=Lambda.min)))
```

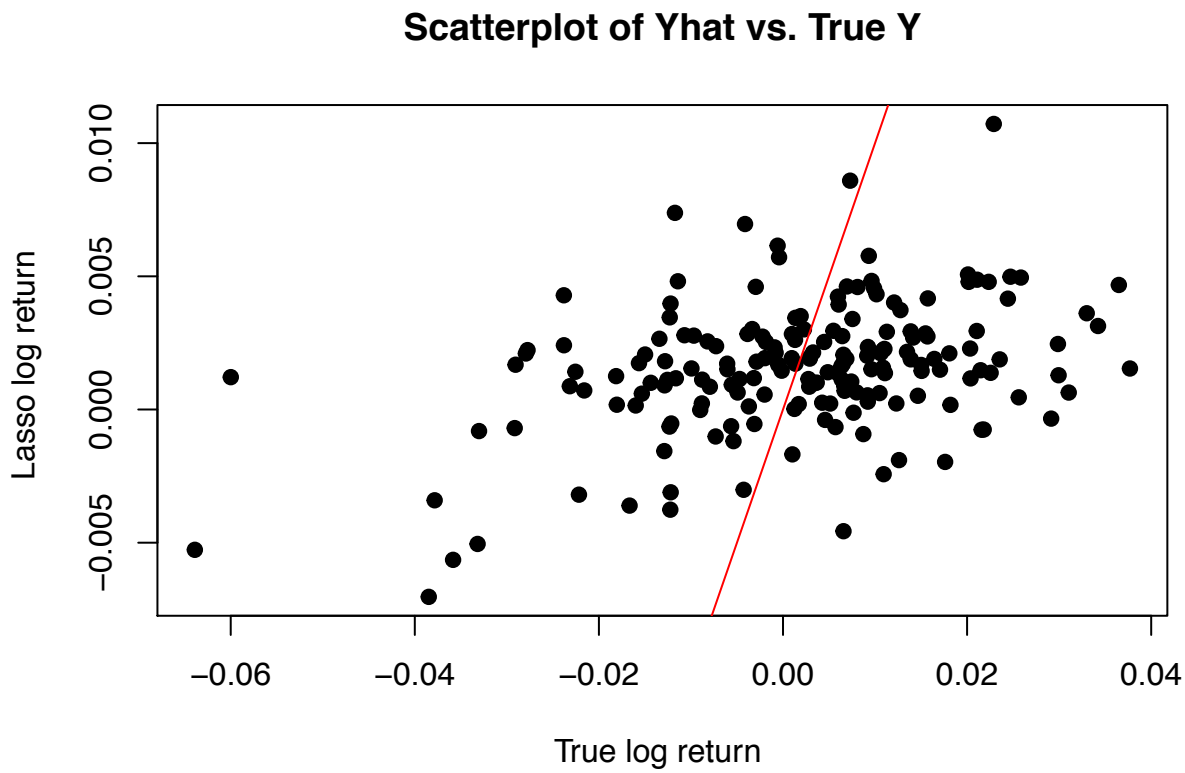
```
## [1] 12
```

```
#Number of predictor when Lambda is 1se
sum(as.logical(coef(fit_lasso,s=Lambda.1se)))
```

```
## [1] 1
```

(c) Number of predictor when we choose Lambda.min is greater than the number of predictor if we choose Lambda.1se. The nonzero coefficients we have are 11 when we choose Lambda.min and 0 when we choose Lambda.1se. Thus, the amount of true signals of this model is way less then 98.

```
set.seed(1)
yhat_lasso = predict(fit_lasso,newx=as.matrix(search.vol),s=Lambda.min)
plot(data$logret,yhat_lasso,main="Scatterplot of Yhat vs. True Y",
   xlab="True log return ", ylab="Lasso log return ", pch=19)
#abline(h=mean(yhat_lasso),v=mean(data$logret),col="blue")
abline(0,1, col="red")
```




```
mean((yhat_lasso-data$logret)^2)
```

```
## [1] 0.0002514228
```

(d) The points scatter along the Y = X line, but with large variance.

```r
set.seed(1)
for (i in 1:nrow(data)){
  yhat_i = predict(fit_lasso,newx=as.matrix(search.vol[i,]),s=Lambda.min)
  if (yhat_i >= 0){
    data$expret[i] = data$logret[i]
  }else{
    data$expret[i] = - data$logret[i]
  }
  #print(yhat_i)
}
sum(data$expret)
```

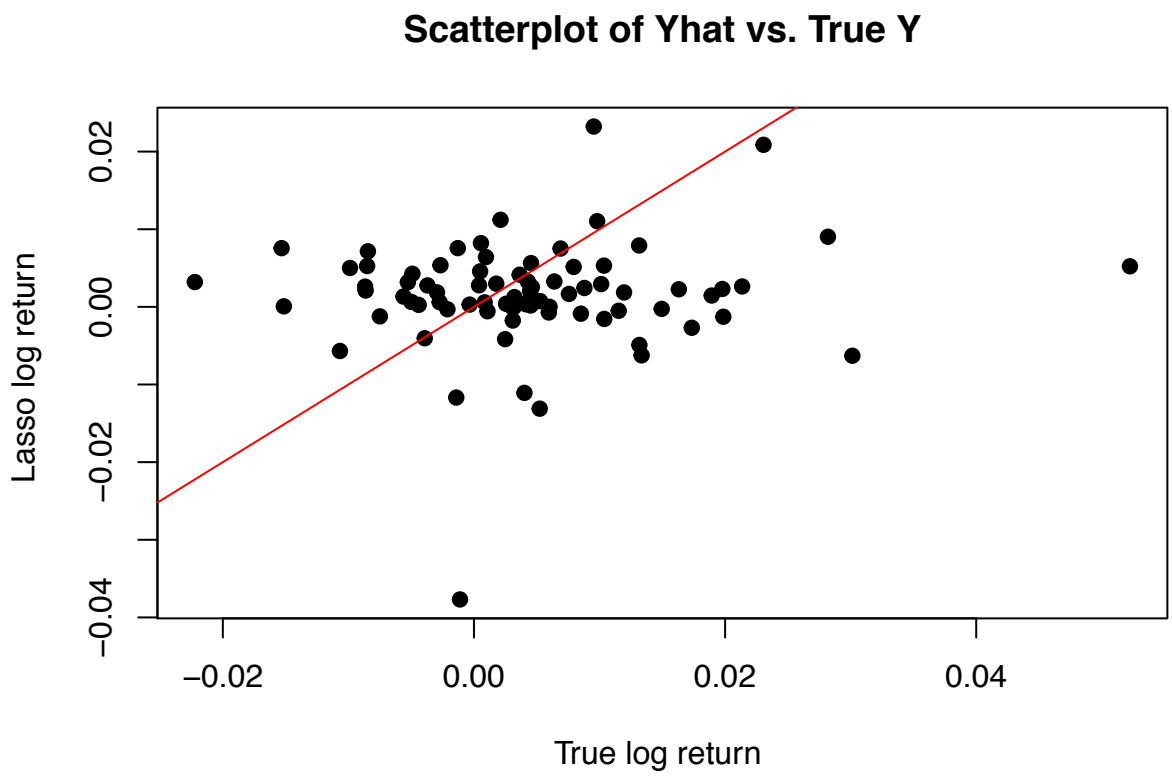
```
## [1] 0.8006779
```

```r
sum(data$logret)
```

```
## [1] 0.2830381
```

(e) My sum of log return of the strategy is 0.8006779, while the return I would have realized if I bought on the first day of the and sold on the last day is 0.2830381 My strategy triumphs.

```r
set.seed(1)
test = read.table('trends_test.csv', sep = ",",header=TRUE)
yhat_lasso_test = predict(fit_lasso,newx=as.matrix(test[,2:(ncol(test)-1)]),s=Lambda.min)
plot(test$logret,yhat_lasso_test,main="Scatterplot of Yhat vs. True Y",
   xlab="True log return ", ylab="Lasso log return ", pch=19)
#abline(h=mean(yhat_lasso),v=mean(data$logret),col="blue")
abline(0,1, col="red")
```




```r
test_search = test[,2:(ncol(test)-1)]
```

```
for (i in 1:nrow(test)){
  yhat_i = predict(fit_lasso,newx=as.matrix(test_search[i,]),s=Lambda.min)
  if (yhat_i >= 0){
    test$expret[i] = test$logret[i]
  }else{
    test$expret[i] = - test$logret[i]
  }
  #print(yhat_i)
}
sum(test$expret)
```

```
## [1] 0.06499931
```

```
sum(test$logret)
```

```
## [1] 0.3461536
```

(f) My sum of log return of the strategy is0.0649993 , while the return I would have realized if I bought on the first day of the and sold on the last day is 0.3461536. My strategy works bad on test data.

Prob 3.

(a) $\hat{y} = Hy = X(X^TX)^{-1}X^Ty$, where $X = UDV^T$

$= (UDV^T)(VDU^TUDV^T)^{-1}VDU^Ty$

$= UDV^TV(D^2)^{-1}V^{-1}VDU^Ty$

$= UU^Ty$

(b) $\hat{y}_{ridge} = X(X^TX+\lambda I)^{-1}X^Ty$, $X = UDV^T$

$= (UDV^T)(VDU^TUDV^T + \lambda I)^{-1}X^Ty$, $I = VV^T = UU^T$

$= (UDV^T)(VD^2V^T + \lambda VV^T)^{-1}X^Ty$

$= (UDV^T)(V(D^2+\lambda I)V^T)^{-1}VDU^Ty$

$= UD(D^2+\lambda I)^{-1}DU^Ty$

(c) $\hat{y}_{ridge} = \underbrace{U}_{n\times p}\underbrace{D(D^2+\lambda I)^{-1}D}_{A \text{ is diagonal } p\times p,\ A_{ii}=\frac{d_i^2}{d_i^2+\lambda}}\underbrace{U^Ty}_{p\times n,\ =n\times n}$

$= UAU^Ty$

$= \sum_{i=1}^{p} A_{ii}u_iu_i^Ty$

$= \sum_{i=1}^{p} \frac{d_i^2}{d_i^2+\lambda}\cdot u_iu_i^Ty$

$\therefore \lambda = 0$, $\hat{y}_{ridge} = \sum_{i=1}^{p} u_iu_i^Ty = UU^Ty$, → linear regression

$\lambda > 0$, $\hat{y}_{ridge}$ decreases as λ increases,

the smaller the d_i, the larger the shrink effect of x.

∴ Shrink in the direction of small variance.